

Product Requirements Document

AI-Powered Blog Companion

Version: 1.0

Status: Development Specification

Primary Stack: MERN + FastAPI + LangChain + LLM + Vector Database

1. Product Overview

The product is a blog publishing platform with an integrated AI chatbot available on each blog's viewing page.

The chatbot is specifically scoped to the **currently viewed blog**. Users can converse naturally with the chatbot to:

- Ask questions about the blog
- Request explanations
- Summarize the blog
- Extract key points
- Generate talking points
- Ask follow-up questions
- Ask questions referring to previous messages
- Explore concepts discussed in the blog

The chatbot must **not answer unrelated questions** or use general model knowledge to answer questions outside the blog.

The chatbot will maintain **temporary conversational memory for a session**.

The existing MERN application remains responsible for normal blog functionality. FastAPI will act as a dedicated AI backend.

2. Goals

Primary Goals

1. Add an AI chatbot to every blog viewing page.
2. Restrict the chatbot's knowledge to the currently viewed blog.
3. Support natural multi-turn conversations.
4. Support temporary conversational memory.

5. Use RAG to ground answers in the blog content.
6. Allow the chatbot to perform multiple tasks through a single chat interface.
7. Avoid re-embedding blogs for every user question.
8. Automatically update the vector representation when a blog is created or modified.
9. Provide a clean API that React can consume.
10. Keep AI functionality isolated from the existing Express backend.

Secondary Goals

- Provide source/chunk references with answers in the future.
 - Make the AI service independently deployable.
 - Make the system extensible to additional AI features later.
-

3. Non-Goals

The first version will NOT:

- Answer general-purpose questions.
 - Search the internet.
 - Use web search.
 - Answer questions about other blogs.
 - Provide a general ChatGPT-like experience.
 - Store permanent conversation history.
 - Require authentication for public blogs.
 - Have separate `/summarize`, `/key-points`, `/explain`, etc. endpoints.
 - Re-embed the blog for every chat message.
 - Replace the existing Express/MongoDB blog backend.
-

4. Existing System

The existing application is a MERN blog platform.

```
graph TD;
  React --> Express;
  Express --> MongoDB;
```

The blog viewing page loads a blog using its `blogId`.

For example:

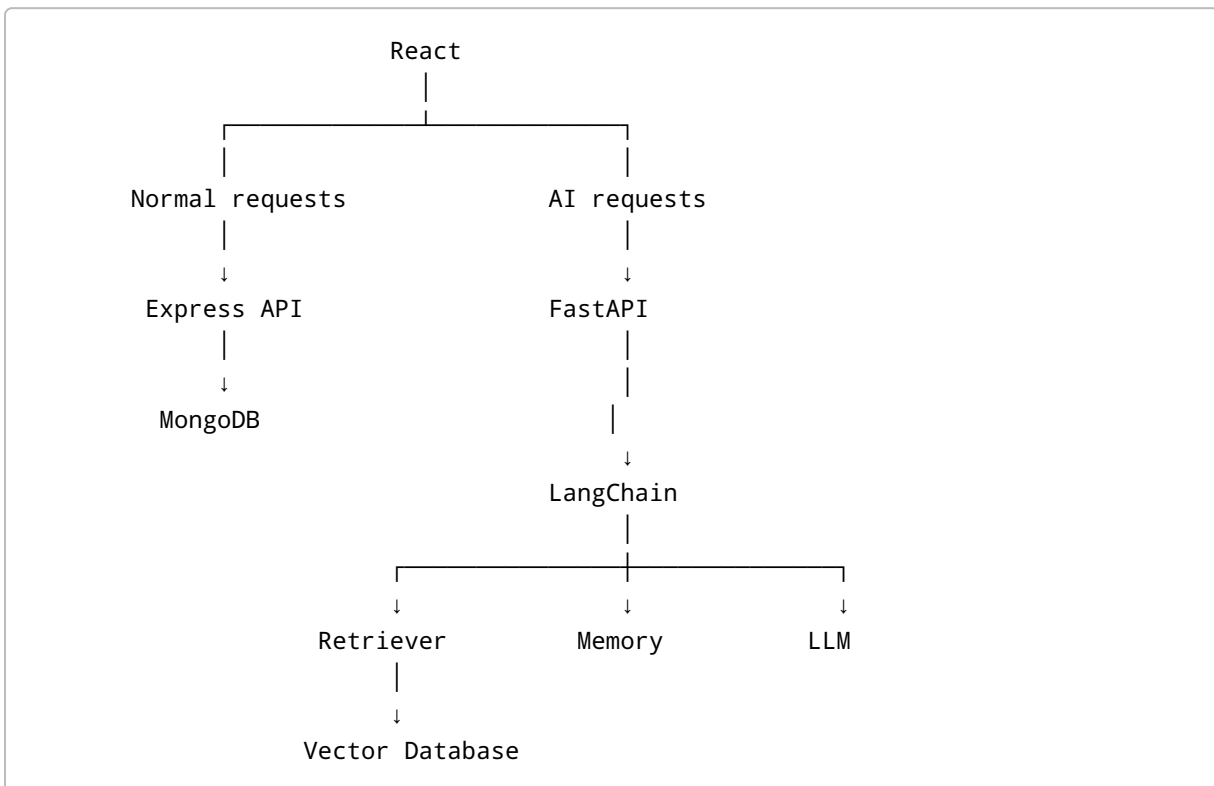
```
/blog/64abc123
```

The frontend already knows:

```
blogId = "64abc123"
```

The AI system will reuse this identifier.

5. Proposed Architecture



Responsibility Separation

React

Responsible for:

- Blog UI
- Chat UI
- Displaying messages
- Maintaining the current `blogId`

- Maintaining/retrieving `sessionId`
- Sending chat requests
- Displaying AI responses

Express

Responsible for:

- Blog CRUD
- Users
- Comments
- Normal application functionality
- MongoDB operations related to the main application

FastAPI

Responsible for:

- AI endpoints
- RAG pipeline
- Vector retrieval
- Conversation memory
- LLM interaction
- Blog ingestion into the vector database

MongoDB

Responsible for:

- Blog documents
- Existing application data

It may also be used by FastAPI to retrieve blog content during ingestion.

Vector Database

Responsible for:

- Blog chunks
- Embeddings
- Similarity search
- Blog-specific retrieval

LLM

Responsible for:

- Understanding user intent

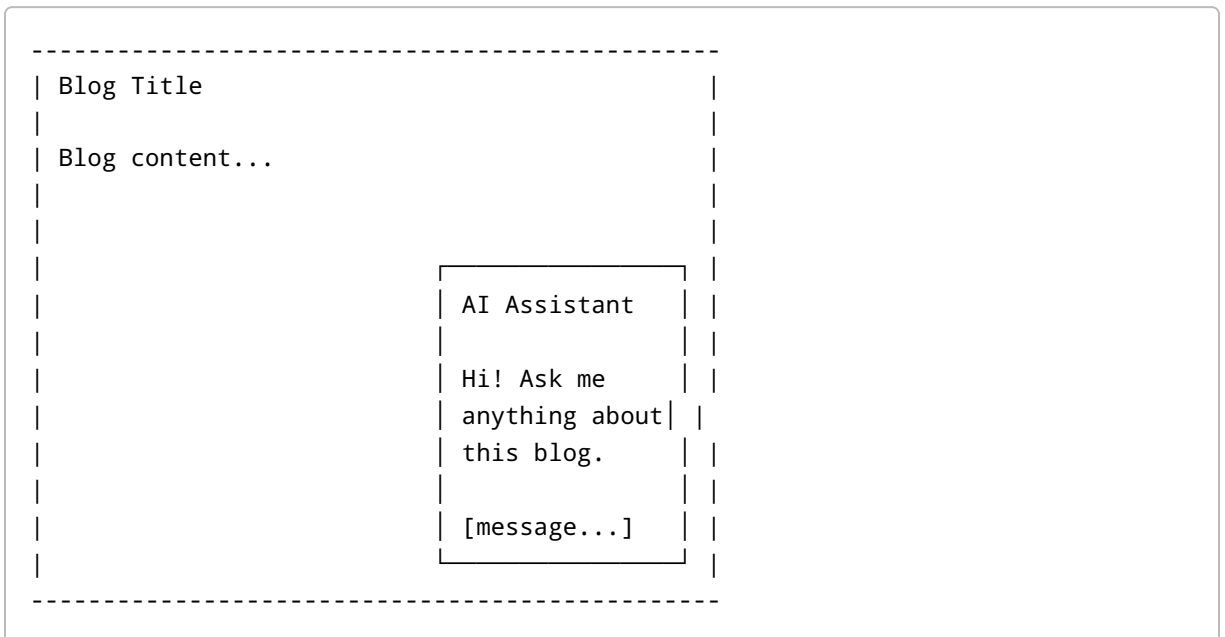
- Generating answers
- Summarization
- Explanations
- Talking points
- Conversational responses

6. Core User Experience

A user opens:

/blog/abc123

They see:



The chatbot is contextualized to that blog.

Example:

User:
What is this article mainly arguing?

AI:
The article makes three main arguments...

Then:

User:
Explain the second one in simpler terms.

AI:
The second argument means...

Then:

User:
Give me some talking points for discussing it.

AI:
Here are several points you could discuss...

All of these use the same `/chat` endpoint.

7. Chatbot Scope

The chatbot's knowledge scope is:

CURRENT BLOG ONLY

The current blog is identified by:

`blog_id`

The chatbot must never retrieve chunks belonging to another blog.

Every vector document must therefore contain metadata:

```
metadata = {  
  "blog_id": blog_id  
}
```

Example:

```
Chunk 1
metadata:
  blog_id = abc123

Chunk 2
metadata:
  blog_id = abc123

Chunk 3
metadata:
  blog_id = abc123
```

Retrieval must be filtered by the current `blog_id`.

8. RAG Architecture

Ingestion

Blog content is processed when the blog is created or updated.

```
Blog
↓
Extract content
↓
Document
↓
Text Splitter
↓
Chunks
↓
Embeddings
↓
Vector Database
```

Each chunk receives metadata:

```
{
  "blog_id": "...",
  "chunk_id": "...",
}
```

Optional metadata:

```
{  
  "blog_id": "...",  
  "chunk_id": "...",  
  "title": "...",  
  "section": "...",  
}
```

9. No Re-Embedding on Every Question

The system must NOT do this:

```
User question  
↓  
Load blog  
↓  
Split blog  
↓  
Embed blog  
↓  
Store vectors  
↓  
Retrieve  
↓  
LLM
```

Instead:

```
BLOG CREATED / UPDATED  
↓  
Split  
↓  
Embed  
↓  
Vector DB
```

Then every chat request does:


```
User question
↓
Query embedding
↓
Vector DB similarity search
↓
Top relevant chunks
↓
LLM
```

This keeps chat requests fast and avoids unnecessary embedding costs.

10. Blog Update Handling

When a blog is created:

```
POST /blogs
↓
Express
↓
MongoDB
↓
AI ingestion
↓
Vector DB
```

When a blog is updated:

```
PUT /blogs/:id
↓
MongoDB
↓
Re-ingest blog
↓
Remove/replace old vectors
↓
Create new vectors
```

The system must avoid having both stale and current versions of the same blog in retrieval.

11. AI Chat API

The primary AI endpoint is:

```
POST /chat
```

Request

```
{
  "blog_id": "abc123",
  "session_id": "xyz789",
  "message": "What are the main arguments?"
}
```

Fields

`blog_id`

Identifies the blog currently being discussed.

Required.

`session_id`

Identifies the temporary conversation.

Required.

`message`

The user's current message.

Required.

12. Chat Response

Basic response:

```
{
  "answer": "The article presents three main arguments..."
}
```

Future response:

```
{
  "answer": "The article presents three main arguments...",
  "sources": [
    {
      "chunk_id": "chunk_12",
      "section": "Introduction"
    }
  ]
}
```

The initial implementation may return only `answer`.

13. Session Management

Each conversation receives a `session_id`.

The React frontend can generate and store the session identifier in `localStorage`.

Example:

```
let sessionId = localStorage.getItem("chat_session");

if (!sessionId) {
  sessionId = crypto.randomUUID();
  localStorage.setItem("chat_session", sessionId);
}
```

The actual conversation messages should be maintained server-side rather than relying on `localStorage` as the source of truth.

14. Temporary Memory

The first implementation will use temporary server-side memory.

Conceptually:

```
session_id
  ↓
Chat History
  ↓
Messages
```

Example:

```
session_123

User:
What is the article about?

AI:
It discusses...

User:
Explain the second point.

AI:
The second point...
```

The memory disappears when the server/session storage is cleared.

No permanent chat-history database is required for V1.

15. Memory vs RAG

These are separate concepts.

Memory

Answers:

What were we talking about?

Example:

User:
Explain the second argument.

The model uses previous conversation context to understand what "second argument" refers to.

RAG

Answers:

What does the actual blog content say?

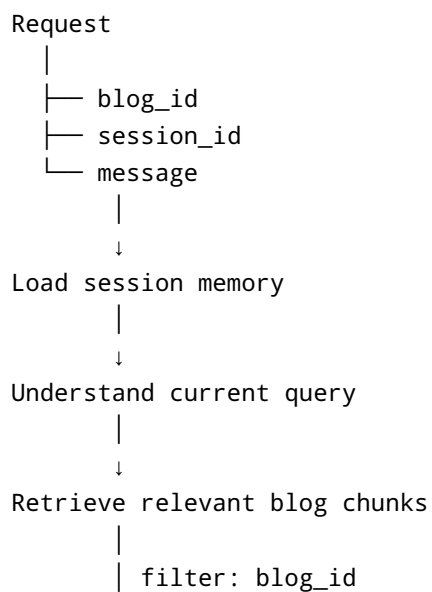
RAG retrieves the relevant blog chunks.

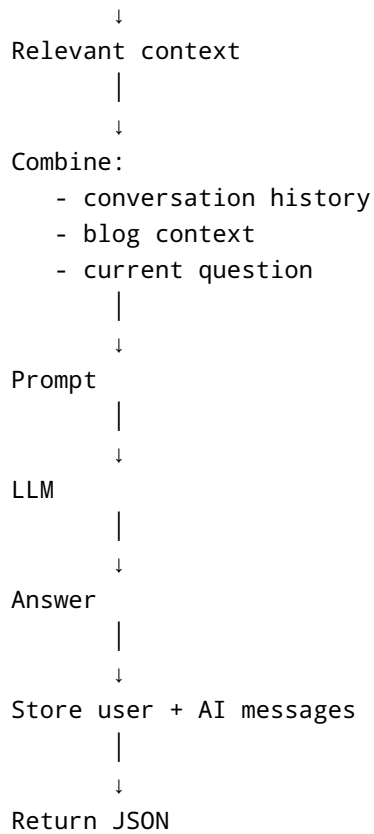
The final prompt uses both:

Conversation History
+
Retrieved Blog Context
+
Current User Message

16. Chat Processing Pipeline

Every `/chat` request follows:





17. Handling Follow-Up Questions

The chatbot must support references such as:

User:
What are the three main points?

AI:
1. ...
2. ...
3. ...

User:
Explain the second one.

AI:
The second point means...

Memory provides the conversational context.

RAG still performs retrieval because the second question may require actual blog information.

18. Handling Summarization

Summarization does NOT have a separate endpoint.

The user simply asks:

Summarize this blog in 5 points.

The same:

POST /chat

endpoint handles it.

The LLM determines the requested operation from the user's message.

19. Handling Talking Points

Likewise:

Give me talking points for discussing this article.

goes through:

POST /chat

The system retrieves relevant blog context and asks the LLM to produce the requested format.

20. Handling Unrelated Questions

Example:

User:
Who won the latest World Cup?

The system must refuse.

Expected behavior:

I can only answer questions related to this blog.

The system should not use the LLM's general pretrained knowledge to answer.

21. Defense Against Unrelated Questions

The restriction should be implemented using multiple mechanisms.

Mechanism 1 — Vector filtering

Only retrieve:

```
blog_id == current blog
```

Mechanism 2 — Retrieval relevance

If retrieved chunks are not sufficiently relevant, do not generate an answer from general model knowledge.

Mechanism 3 — System prompt

The system prompt should explicitly state:

You are an AI assistant for the current blog.

Answer only using information available
in the provided blog context and relevant
conversation history.

You may answer questions about the blog,
summarize it, explain concepts from it,
extract key points, generate discussion points,
and answer contextual follow-up questions.

If the requested information cannot be supported by the blog context, do not answer using outside knowledge.

Tell the user that you can only answer questions related to the current blog.

22. Important Limitation

Prompt instructions alone are **not a security boundary**.

The actual restriction must primarily come from the architecture:

```
blog_id filter
+
retrieval relevance
+
prompt restriction
```

This reduces the chance that the LLM simply answers an unrelated question using its pretrained knowledge.

23. Frontend Responsibilities

The React chatbot component should maintain:

```
messages
blogId
sessionId
input
loading
error
```

Example state:

```
const [messages, setMessages] = useState([]);
const [input, setInput] = useState("");
const [loading, setLoading] = useState(false);
```

The frontend sends:

```
await fetch("http://localhost:8000/chat", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    blog_id: blogId,
    session_id: sessionId,
    message: input
  })
});
```

The frontend displays the returned `answer`.

24. FastAPI Request Schema

Use Pydantic:

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    blog_id: str
    session_id: str
    message: str
```

Endpoint:

```
@app.post("/chat")
async def chat(data: ChatRequest):
    ...
```

FastAPI handles JSON parsing and validation.

25. FastAPI Service Structure

Initial project:

```
ai-service/  
|  
├─ main.py  
├─ .env  
├─ requirements.txt  
|  
├─ schemas/  
│   └─ chat.py  
|  
├─ services/  
│   ├── chat.py  
│   ├── rag.py  
│   ├── memory.py  
│   └─ ingestion.py  
└─ vectorstore/  
    └─ ...
```

For the first prototype, this can be simplified:

```
ai-service/  
|  
├─ main.py  
├─ rag.py  
└─ .env
```

Refactoring can happen once functionality works.

26. Ingestion API

A second FastAPI endpoint is needed for backend-to-AI ingestion.

```
POST /ingest
```

Request:

```
{  
  "blog_id": "abc123"  
}
```

Its responsibility:

```
blog_id
↓
retrieve blog
↓
process content
↓
split
↓
embed
↓
store/update vectors
```

This endpoint is NOT a user-facing chatbot endpoint.

It is an internal AI ingestion operation.

27. Ingestion Trigger

When Express creates or updates a blog:

```
React
↓
Express
↓
MongoDB
↓
FastAPI /ingest
↓
Vector DB
```

The exact triggering mechanism can initially be a direct HTTP request from Express.

Later, this can be replaced by a queue/event system if required.

28. Vector Store Design

Each vector document should contain:

```
Document(  
  page_content="...",  
  metadata={  
    "blog_id": "abc123",  
    "chunk_id": "abc123_001"  
  }  
)
```

Optional:

```
metadata={  
  "blog_id": "abc123",  
  "chunk_id": "abc123_001",  
  "title": "Introduction to RAG",  
  "section": "Introduction"  
}
```

The vector store can contain chunks from every public blog.

Retrieval is always scoped to the current blog.

29. Blog Deletion

If a blog is deleted:

```
Delete blog from MongoDB  
    ↓  
Delete all vectors  
where blog_id == deleted blog
```

This prevents stale content from remaining in the vector database.

An internal endpoint could eventually be:

```
DELETE /ingest/{blog_id}
```

or ingestion/deletion can be handled internally by the AI service.

30. Error Handling

FastAPI should return appropriate HTTP errors.

Examples:

Missing blog

```
404 Not Found
```

Invalid request

```
422 Unprocessable Entity
```

LLM failure

```
500 Internal Server Error
```

Vector store failure

```
503 Service Unavailable
```

The frontend should display a user-friendly error rather than raw backend errors.

31. CORS

During development:

```
React  
localhost:5173
```

```
FastAPI  
localhost:8000
```

The browser considers these different origins.

FastAPI therefore needs CORS configuration allowing the React development origin.

Production should allow only the actual frontend origin rather than:

```
allow_origins=["*"]
```

unless there is a specific reason to do so.

32. Environment Variables

Secrets must never be hardcoded.

`.env` may contain:

```
LLM_API_KEY=...  
MONGODB_URI=...  
VECTOR_DB_URL=...  
VECTOR_DB_API_KEY=...
```

Python should load these from environment configuration.

The `.env` file must not be committed to Git.

33. Initial Technology Choices

Recommended V1 stack:

```
Frontend  
React  
  
Normal backend  
Express.js  
  
Main database  
MongoDB  
  
AI backend  
FastAPI  
  
AI framework  
LangChain
```

```
LLM
Hugging Face / OpenAI / other provider

Embeddings
Embedding model supported by chosen provider

Vector database
Choose one suitable for the project

Temporary memory
Server-side memory initially
```

The exact LLM and vector store can remain configurable.

34. Session Lifetime

V1 session behavior:

```
Browser
  ↓
localStorage
  ↓
session_id
```

The server maintains:

```
session_id → conversation history
```

The session is temporary.

No permanent conversation history is required.

Future improvement:

```
Redis
```

for shared temporary memory when multiple FastAPI instances are deployed.

35. Security Considerations

Blogs are public, so authentication is not required for reading them.

However:

- API keys must remain server-side.
 - React must never receive LLM/vector database credentials.
 - CORS must be restricted in production.
 - Request sizes should be limited.
 - Chat input should be validated.
 - Rate limiting should eventually be added.
 - The chatbot must not expose internal prompts or credentials.
 - The AI service should not blindly execute arbitrary tools based on user input.
-

36. Performance Requirements

Chat

The target is to avoid unnecessary operations.

A chat request should NOT:

- Load the entire blog from MongoDB.
- Re-split the blog.
- Re-create embeddings.
- Re-insert vectors.

Instead:

```
Question
↓
Query embedding
↓
Filtered vector search
↓
Top-k chunks
↓
LLM
```

Ingestion

Embedding work happens only when:

- Blog is created.
 - Blog is updated.
 - Blog needs to be re-indexed.
-

37. Suggested Retrieval Configuration

Initial implementation:

Top K = 4-6 chunks

This should be tuned experimentally.

Too few:

insufficient context

Too many:

larger prompt
higher cost
more irrelevant information

The exact value is not fixed by the PRD and should be evaluated with real blog content.

38. Chat History Management

Memory should not grow indefinitely.

For V1, keep a bounded number of recent messages.

For example:

last 10-20 messages

or a token-based history limit.

Older conversation can eventually be summarized if long conversations need to be supported.

39. Expected Chat Behavior

Question about blog

User:
What problem is the author trying to solve?

Bot:
The author is trying to solve...

Summary

User:
Give me a concise summary.

Bot:
...

Explanation

User:
Explain the third section like I'm a beginner.

Bot:
...

Talking points

User:
What points could I bring up in a discussion?

Bot:
...

Follow-up

User:
Why does that matter?

Bot:
...

Unrelated

User:
What's the weather today?

Bot:
I can only answer questions related to this blog.

40. Acceptance Criteria

The feature is considered successful when:

AC1 — Blog Context

Given a user viewing Blog A, the chatbot can answer questions using Blog A's content.

AC2 — Cross-Blog Isolation

Given Blog A and Blog B in the vector database, a user viewing Blog A cannot retrieve Blog B's content.

AC3 — No Re-Embedding

Sending multiple chat messages does not re-embed the blog.

AC4 — Memory

The chatbot can correctly understand reasonable follow-up references within a session.

AC5 — Multi-purpose Chat

The same `/chat` endpoint supports questions, summaries, explanations, key points, and talking points.

AC6 — Unrelated Questions

The chatbot refuses questions that cannot be supported by the current blog.

AC7 — Blog Updates

Updating a blog updates its corresponding vectors.

AC8 — Blog Deletion

Deleting a blog removes its vectors.

AC9 — Session Isolation

Conversation history from Session A must not appear in Session B.

AC10 — API Validation

Invalid chat requests are rejected by FastAPI/Pydantic.

AC11 — Frontend Integration

React can send:

```
{
  "blog_id": "...",
  "session_id": "...",
  "message": "..."
}
```

and receive:

```
{
  "answer": "..."
}
```

AC12 — Secrets

LLM and vector database credentials are never exposed to the React client.

41. V1 Development Plan

Phase 1 — FastAPI Foundation

Build:

```
FastAPI
Pydantic
POST /chat
```

Initially return a dummy response.

```
React → FastAPI → JSON
```

Phase 2 — Connect LangChain

Replace dummy response with:

```
FastAPI
↓
LangChain
↓
LLM
↓
response
```

No RAG yet.

Phase 3 — Blog Ingestion

Build:

```
POST /ingest
```

and implement:

```
Blog
↓
Document
↓
Splitter
↓
Embeddings
↓
Vector DB
```

with:

```
blog_id
```

metadata.

Phase 4 — RAG Chat

Implement:

```
/chat
↓
Retriever
↓
blog_id filter
↓
Context
↓
LLM
```

At this point the chatbot understands the blog.

Phase 5 — Memory

Add:

```
session_id
↓
temporary chat history
```

↓
RAG + memory

Now multi-turn conversations work.

Phase 6 — Restriction

Add:

```
retrieval relevance
+
blog-only system prompt
+
no-context refusal
```

Test extensively with unrelated questions.

Phase 7 — React Chat UI

Build:

```
Chat button
↓
Chat window
↓
Messages
↓
Input
↓
Loading state
↓
POST /chat
↓
AI response
```

Phase 8 — Blog Lifecycle

Connect:


```
Create blog
  ↓
Ingest

Update blog
  ↓
Re-ingest

Delete blog
  ↓
Delete vectors
```

Phase 9 — Production Hardening

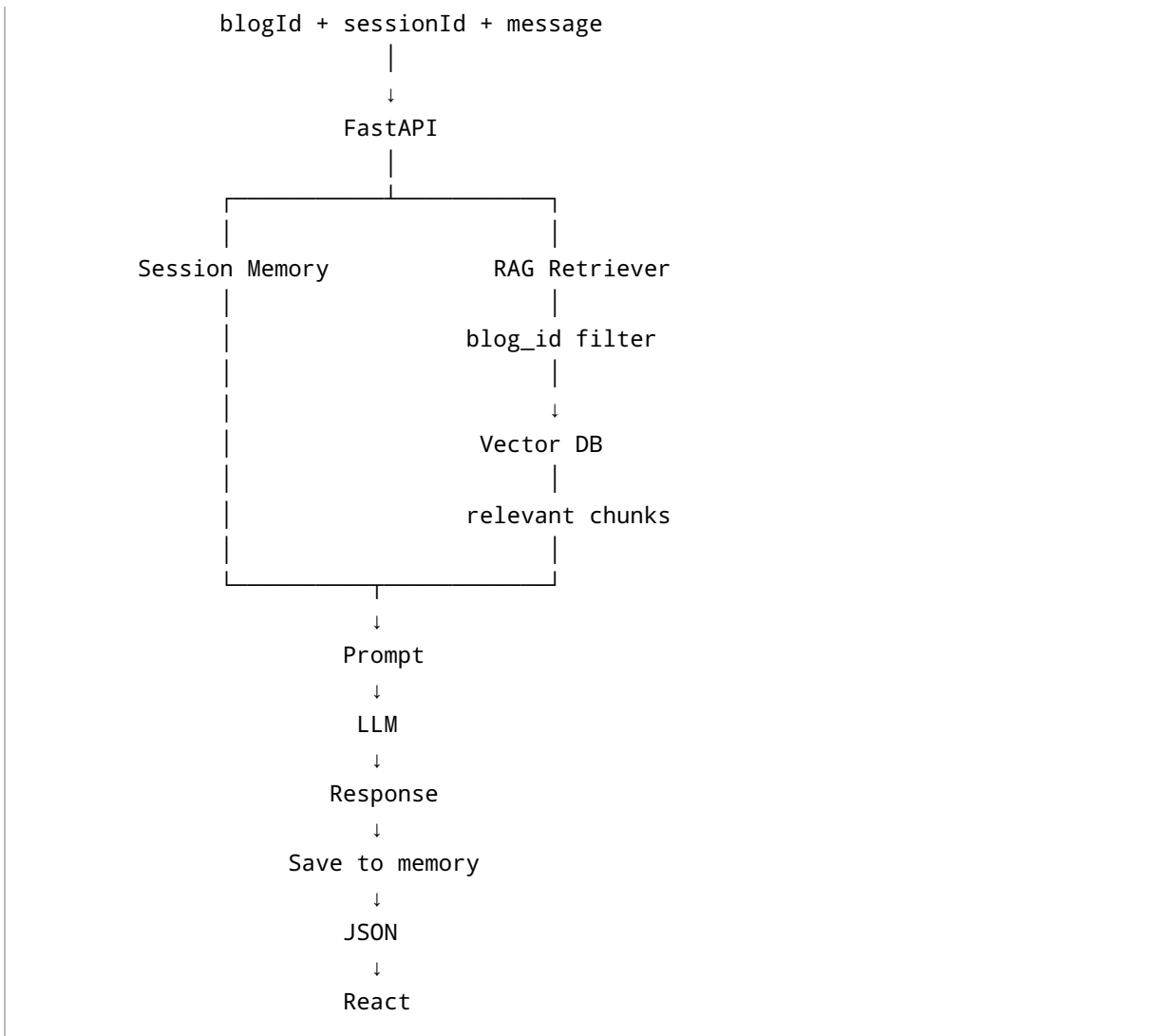
Add:

- Rate limiting
- Better error handling
- Logging
- Request limits
- Redis if required
- Production CORS
- Monitoring
- Deployment
- Streaming responses

42. Final System

The completed system should behave like this:

```
USER
  |
  ↓
React Blog Page
  |
Current blogId
  |
  ↓
AI Chat Window
  |
  |
```



The core principle is:

The blog is indexed once when it changes. Every chat message performs a filtered retrieval against those existing vectors, combines the retrieved blog context with temporary conversation memory, and sends that to the LLM.

This gives you a clean separation between your **MERN application**, **AI service**, **RAG knowledge base**, and **temporary conversational state**.